VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

WEEK 5 INFORMED SEARCHES

Informed search involves utilizing specific knowledge pertaining to a problem to make the search process more efficient and targeted. This is achieved by employing heuristics to evaluate which nodes or states to expand during the search. As a result, it enables the exploration of more promising areas in the search space, ultimately leading to a more optimal solution.

When discussing the basic concepts of search algorithms, there are a few key terms to keep in mind. The first is Search Space, refers to the database or space in which the search is conducted. The next important concept is the Initial State, which is where the search begins. Next is goal State, which is the desired outcome of the search, such as a destination in a Google Maps search. The goal test is then used to determine whether the current state is, in fact, the goal state.

In addition to these core concepts, Path Cost, which is a numerical measurement of the cost required to achieve the goal. Finally, consider the Heuristic Function, which is used to measure the proximity of the current state to the goal state and determine the best path forward based on the path cost. By keeping all of these concepts in mind, effectively understand and utilize search algorithms to achieve desired outcomes.

The different types of informed searches are

1. Best First Search f(n) = h(n) (Breadth First Search +Depth First Search)
2. A* Search f(n) = g(n)+h(n) (Greedy Best first Search + Uniform Cost Search)
3. Local Search = Optimization
4. Hill Climbing
5. Simulated Annealing
6. Genetic Algorithm
7. Greedy Best First Search

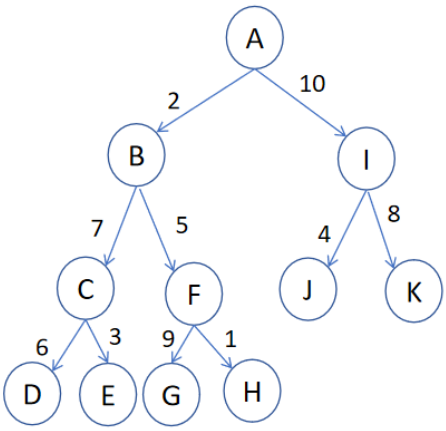
The greedy best first search works as follows : expand the node that is closest to the goal. The evaluation function is written as f(n) = h(n). The greedy best first search is the combination of Breadth First Search and Depth First Search.

The complexity analysis is

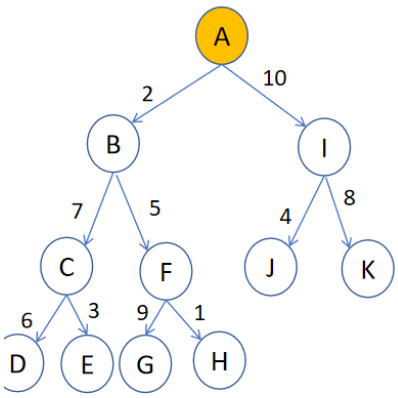
- Time Complexity: O(bm)
- Space Complexity: O(bm) m is the maximum depth of the search space
- Complete: Incomplete, even if the given state space is finite
- Optimal: Not optimal

Example 1:

Reach G from A applying Greedy Best first Search. The heuristic values are given.



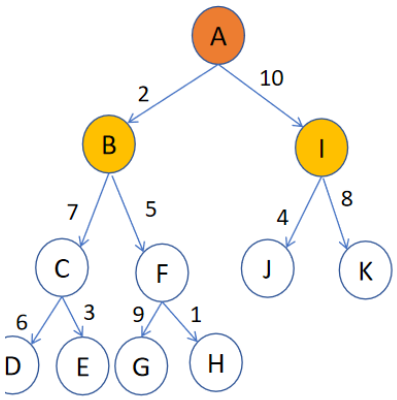
Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10



Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10

- Two lists open and close are created
- Initally only node A is present in the open list and closed list is empty

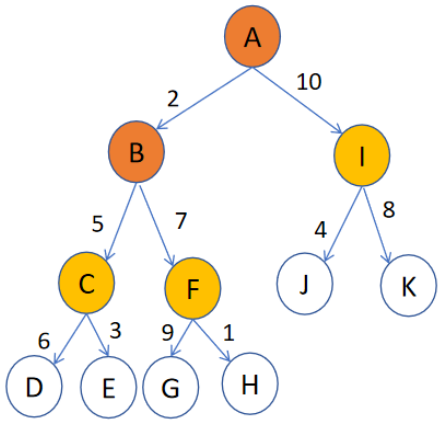
Open : [A]
Closed: [ ]



Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10

- First iteration: Pop node A and move it to the closed list
- Explore A and add them to open
- B I will be explored

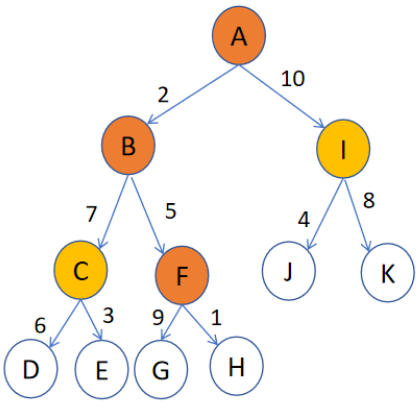
Open : [B I]
Closed: [A]



Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10

- Second iteration: Heuristic value of nodes B and I are compared
- B has lower heuristic it is poped and moved to the closed list
- Neighboring nodes of B are pushed to the open list

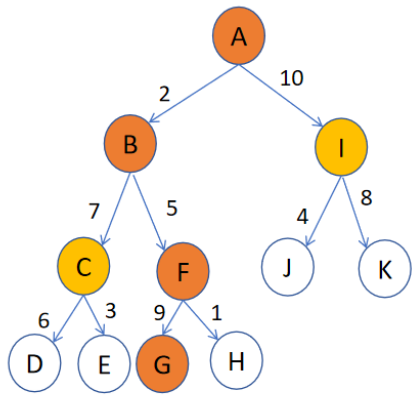
Open : [C F I]
Closed: [A B]



Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10

- Third iteration: Heuristic value of nodes C F and I are compared
- F has lower heuristic, it is poped and moved to the closed list
- Neighboring nodes of F are pushed to the open list

Open : [G H I C]
Closed: [A B F]



Nodes	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
G	0
H	13
I	12
J	10

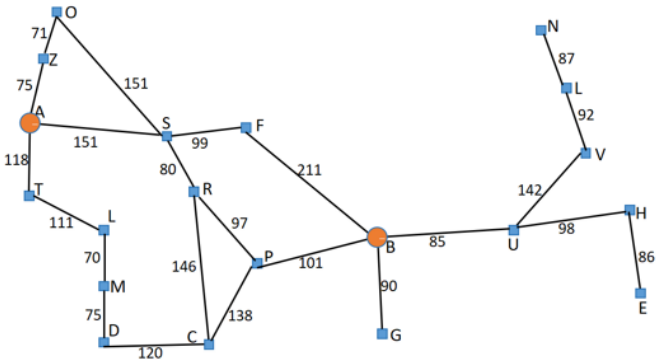
- Fourth iteration: Heuristic value of nodes G H C and I are compared
- G has lower heuristic, it is poped and moved to the closed list
- Goal state reached

Open : [H I C]

Closed: [A B F G]

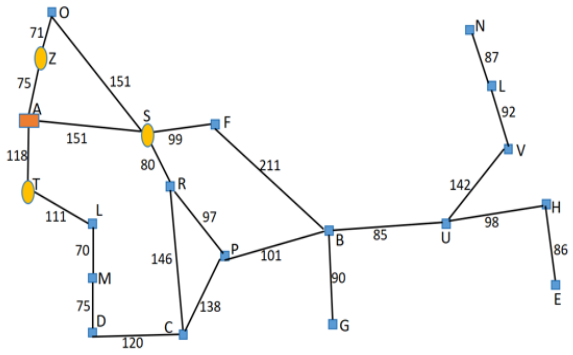
Example 2:

For the given graph, travel from node A to B applying Greedy Best First search. The straight Line distance is given.



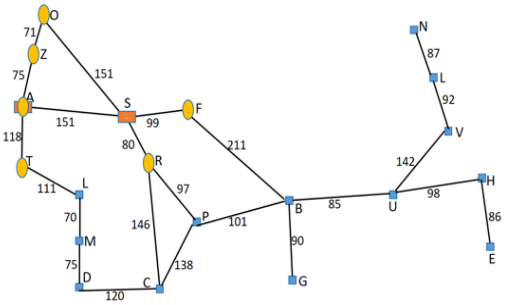
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

Visit A and explore A



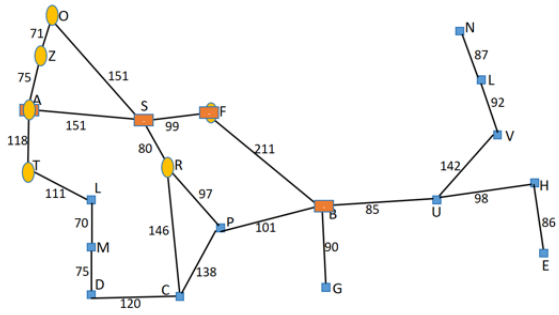
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- A connected with S- Z - T
- Straight line distance for S-253 Z-374 T-329
- Shortest distance is S



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- Visit S and explore S
- S connected with A-F-O-R
- Straight line distance for A-366 F-176 O-380 R-193
- Shortest distance is F



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- Visit F and explore F
- F connected with S-B
- Straight line distance for S-253 B-0
- Shortest distance is B
- Goal reached
- Path is ASFB

A* Search

A* Search is the combination of Uniform cost search and Greedy best first search.

It is a very useful algorithm for finding the shortest path between two points. Specifically, it is commonly used for map traversal to determine the shortest route. Originally, A* was created to aid in building a robot that could plot its own course, but it has since become a popular choice for graph traversal as well. One key feature of A* is that it prioritizes looking for shorter paths, which makes it an optimal and complete algorithm. An optimal algorithm will find the best possible solution, while a complete algorithm considers all possible outcomes. Additionally, A* utilizes weighted graphs, meaning it considers the cost associated with each possible path and ultimately selects the one with the lowest cost. This allows for efficient and effective determination of the best path in terms of distance and time.

From a given starting cell, get to the target cell as quickly as possible. It is the sum of two variables' values that determines the node it picks at any point in time. At each step, it picks the node with the smallest value of 'f' (the sum of 'g' and 'h') and processes that node/cell. 'g' and 'h' is defined as simply as possible below:

$f(n) = g(n)+h(n)$

'g' is the distance it takes to get to a certain square on the grid from the starting point, following the path we generated to get there.

'h' is the heuristic, which is the estimation of the distance it takes to get to the finish line from that square on the grid.

Approximation Heuristics

To determine h, there are typically three approximation heuristics:

1. Manhattan Distance

The Manhattan Distance is the total of the absolute values of the discrepancies between the x and y coordinates of the current and the goal cells.

2. Diagonal Distance

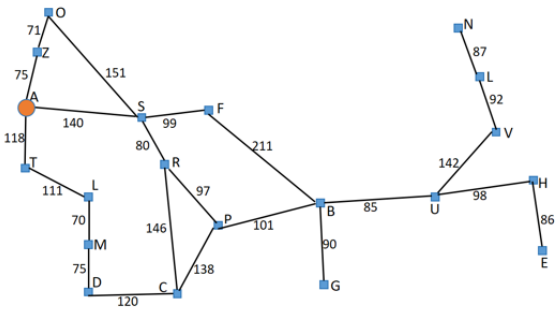
It is nothing more than the greatest absolute value of differences between the x and y coordinates of the current cell and the goal cell.

3. Euclidean Distance

The Euclidean Distance is the distance between the goal cell and the current cell using the distance formula.

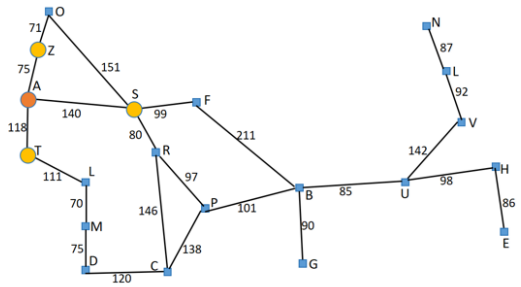
Example 1 :

For the given graph, travel from node A to B applying A* Search. The straight Line distance is given.



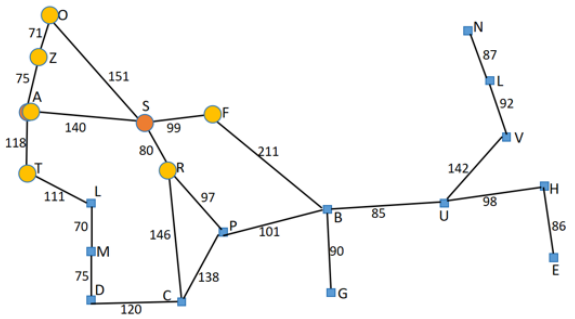
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

Visit A and explore A



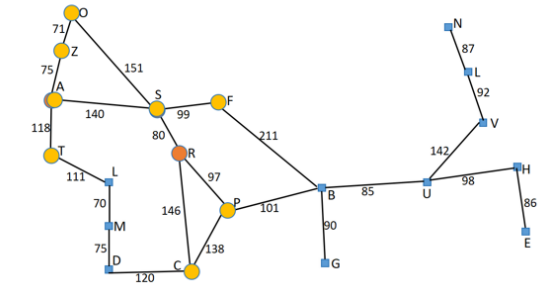
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- A connected with S- T - Z
- f(n)= g(n)+h(n)
- value for
  - AS = 393 (140+253)
  - AT = 447 (118+329)
  - AZ = 449 (75+374)
- Shortest distance : AS
- Node to visit : S



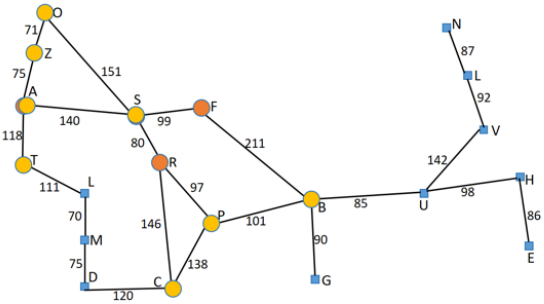
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- S connected with F-R-A-O
- f(n)= g(n)+h(n)
- value for
  - ASA = 646 (140+140+366)
  - ASF = 415 (140+99+176)
  - ASO = 671 (140+151+380)
  - ASR = 413 (140+80+193)
- Shortest distance : ASR
- Node to visit : R



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

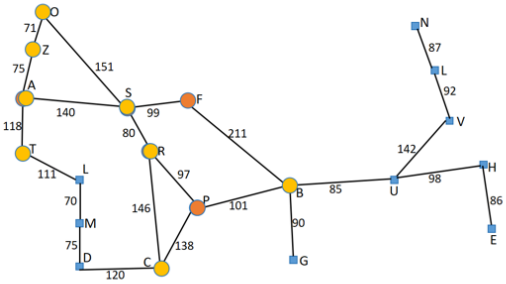
- R connected with P-C-S
- f(n)= g(n)+h(n)
- value for
  - ASF = 415 (140+99+176)
  - ASRS = 553 (140+80+80+253)
  - ASRC = 526 (140+80+146+160)
  - ASRP = 417(140+80+97+100)
- Shortest distance : ASF
- Node to visit : F



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- F connected with S- B
- f(n)= g(n)+h(n)
- value for
  - ASRP = 417(140+80+97+100)
  - ASFB = 450 (140+99+211+0)
  - ASFS = 591(140+99+99+253)

  Goal State reached
- Shortest distance : ASRP
- Node to visit : P



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- P connected with R- B
- f(n)= g(n)+h(n)
- value for
  - ASFB = 450(140+99+211+0)
  - ASRPB = 418(140+80+97+101+0)
  - ASRPC = 615(140+80+97+138+160)
  - ASRPR = 607(140+80+97+97+193)
- Shortest distance : ASRPB
- Node to visit : B
- Goal acheived with least cost

Example 2 :

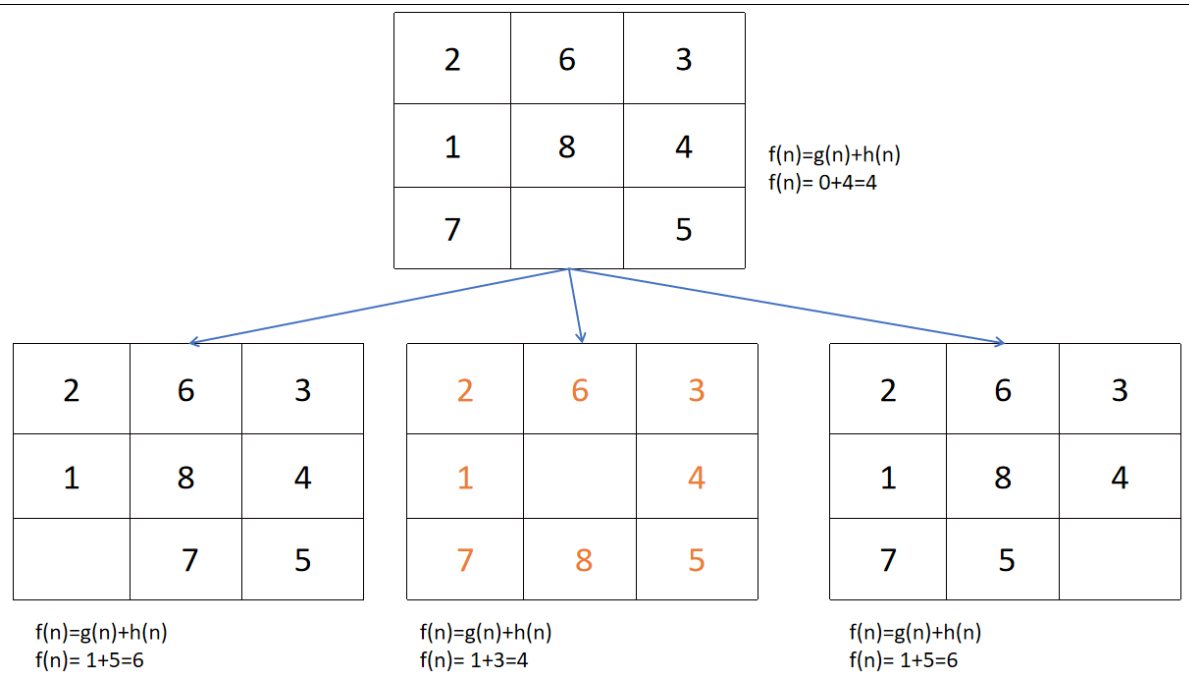
For 8-puzzle problem, given the start state arrive to goal state applying A* search

Start State

2	6	3
1	8	4
7		5

Goal State

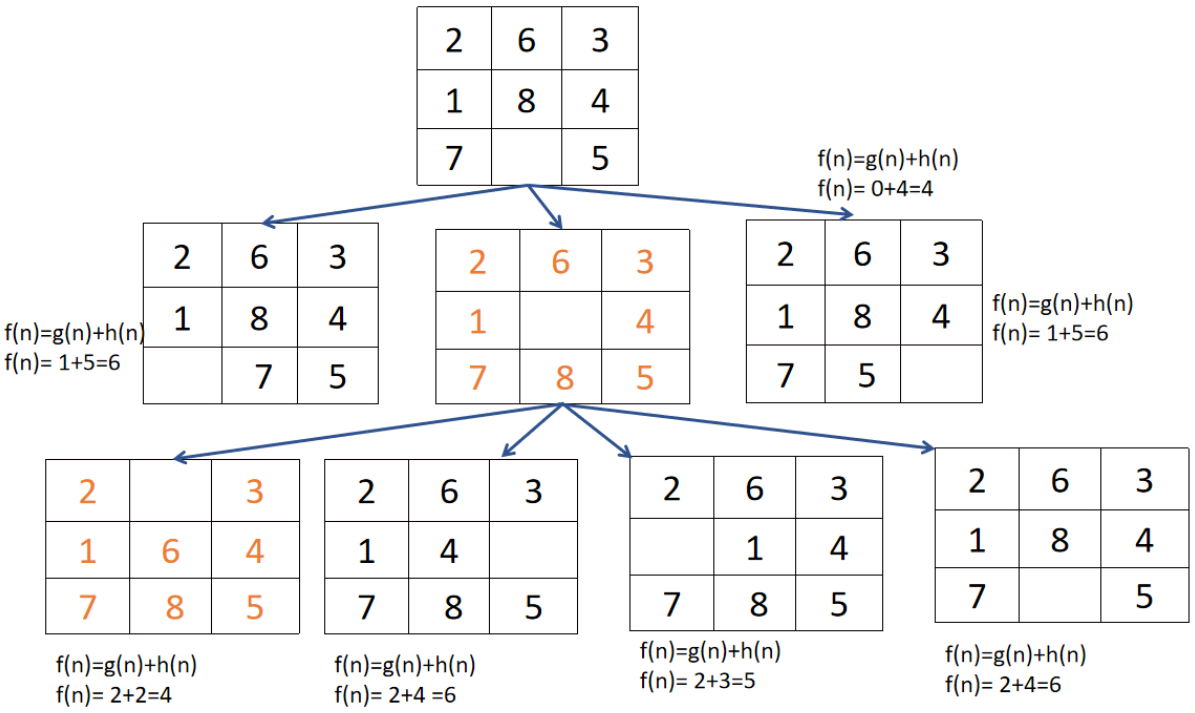
1	2	3
	6	4
7	8	5



Explore all the possibilities by sliding in through the blank space.

Calculate f(n) for each move

Choose the minimum f(n) and explore it

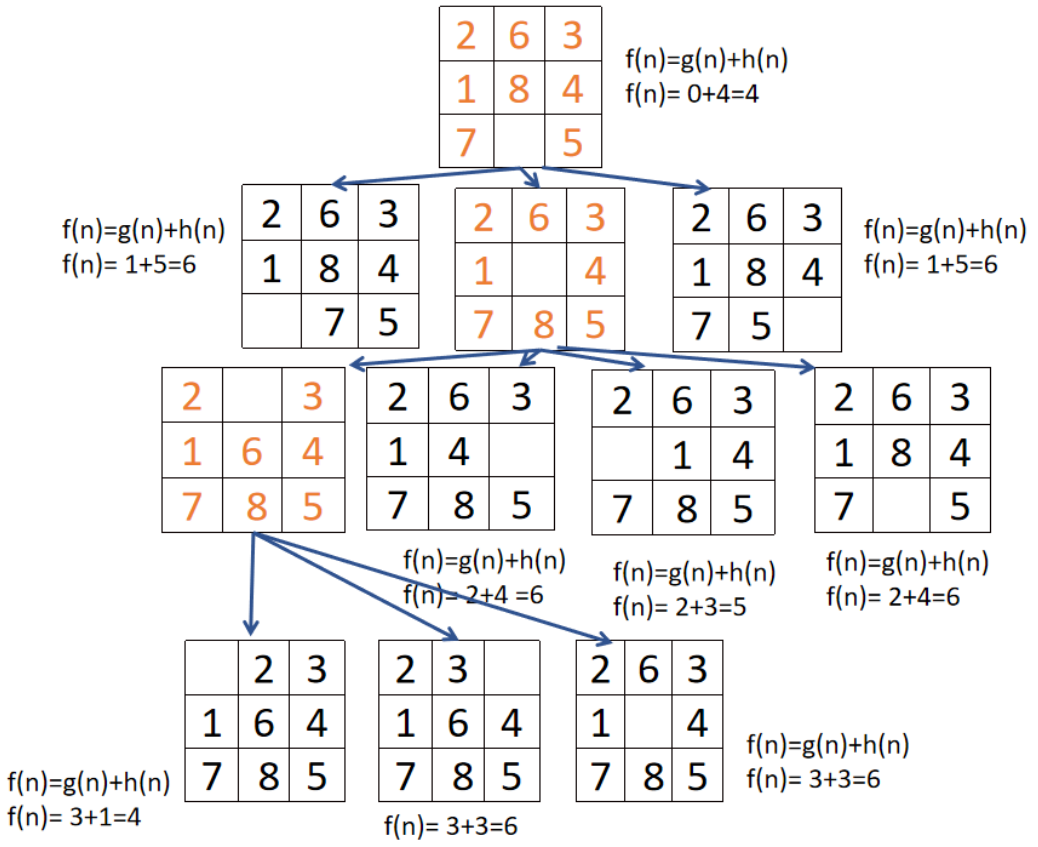


Explore all the possibilities by sliding in through the blank space of the chosen tile

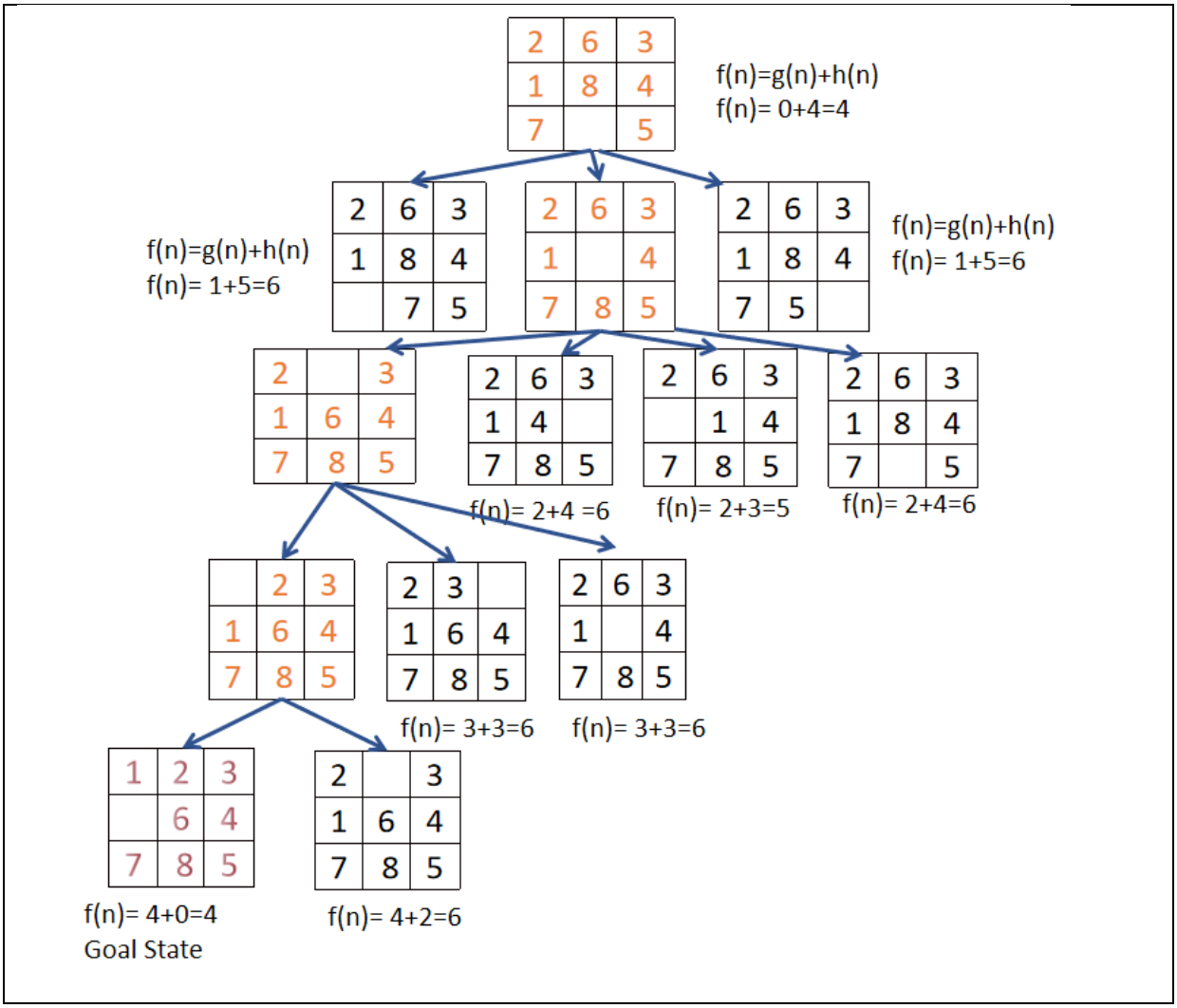
Calculate f(n) for each move

Choose the minimum f(n) and compare it with the previous level.

If the previous level have lesser f(n) choose that else continue with this level of minimum value



Repeat the steps



The Optimality of A* can be measured in terms of conditions namely

Admissible when applied on tree. It never overestimates the cost to reach the goal. The straight-line obtained is the shortest path between any two points.

Consistent when applied on graph search. A heuristic h(n) is consistent if, every node n and every successor n' of n generated by any action a, the estimated cost of reaching the goal from n is no greater than the step cost of getting to n'plus the estimated cost of reaching the goal from n'h(n) $\leq$ c(n, a, n') + h(n')

Triangle inequality: Each side of a triangle cannot be longer than the sum of the other two sides. Here, the triangle is formed by n, n' and the goal G_n closest to n.

Advantages of A* Algorithm

1. Guarantees the optimal path when used with appropriate heuristics.
2. Efficient and can handle large search spaces by effectively pruning unpromising paths.
3. Easily tailored to accommodate different problem domains and heuristics.

4. Flexible and adaptable to varying terrain costs or constraints. Additionally, it is widely implemented and has a vast amount of resources and support available.

Disadvantages of A* Algorithm

- Computationally expensive in certain scenarios, especially when the search space is extensive and the number of possible paths is large.
- Consume significant memory and processing resources.
- Heavily relies on the quality of the heuristic function. If the heuristic is poorly designed or does not accurately estimate the distance to the goal, the algorithm's performance and optimality may be compromised.

Variants of A* search

The variants of A* can be represented in a simple formula.

$f=w1*g+w2*h$

w1 = 1 w2 = 0 Dijkstra's Algorithm search without a heuristic

w1 = 0 w2 = 1 Greedy best-first search with low heuristic values.

w1 = 1 w2 = 1 A*

w1 = 1 w2 = 10 Weighted A* find a solution more quickly

The different variants of A* search are :

1. Beam Search
2. Weighted A*
3. Dynamic A* and Lifelong Planning A*
4. Jump Point Search

Beam Search

Beam search uses breadth first search. At each level of the tree, it generates all successors of the states at the current level, sorting them in increasing order of heuristic cost. It stores a predetermined limit (β) and gives optimal solution for it. If the set becomes too large, the node with the worst chances of giving a good path is dropped. Then it returns the first solution found. The main drawback of beam search is that the set taken should be sorted.

The complexity analysis of beam search are

- Time Complexity : O(B*m) B is the beam width m is the maximum depth of search tree
- Space Complexity : O(B*m)

- Completeness: Complete
- Optimality: Not optimal

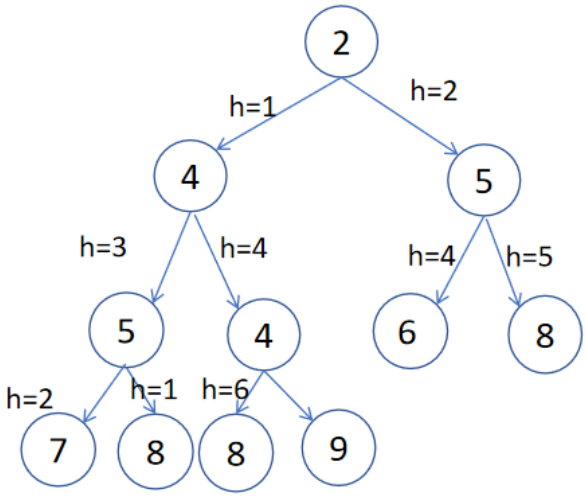
The variants of beam search are

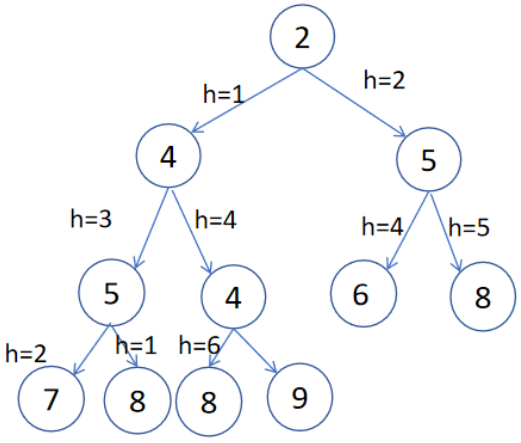
1. Beam stack search : combination of Beam Search and DFS
2. Beam search using limited discrepancy backtracking : combination of Beam Search and Depth Limited Search

The beam search is mainly used in large systems with insufficient amount of memory to store the entire search tree. It is also used in machine translation systems such as speech recognition system

Example 1:

Reach goal state 9 from root node 2 with β = 3 by applying Beam Search





Step 1: OPEN= {2}

Step 2: OPEN= {4, 5}

Step 3: OPEN= {5, 4}

Step 4: OPEN= {5}

Step 5: OPEN= {8}

Step 6: OPEN= {}

Goal Node not visited

Weighted A*

Multiply the heuristic by some constant factor to make A* run faster. This search use some weight w > 1 to increase the heuristic's effect

$f(n)=g(n)+w*h(n)$

If the heuristic is too low on some parts of the path use dynamic weighting

$f(n)=g(n)+w(n)*h(n)$

Guarantee bounded sub-optimality

Weighted A* usually fasten the search

Dynamic A*

Dynamic A* is used when there is no complete information available. when not all the information are available, A* tend to make mistakes. The Dynamic A* correct those mistakes without taking much time.

Dynamic A* finds the shortest path from its current coordinates to the goal coordinates. When obstacles are detected, this search finds the shortest path from its current coordinates to the goal coordinates. The process is repeated until goal state. When a new obstacle is detected, new planning will be done.

The variants of dynamic A* are

- D*: Informed incremental search
- Focused D*: A* + D*
- D* Lite : Similar to LPA*, search starts from goal state and move towards the start state

Lifelong Planning

Lifelong Planning A* is applied when the costs are changing. With A*, the path may be invalidated by changes to the map. LPA* can re-use previous A* computations to produce a new path.

Memory Bound Heuristics

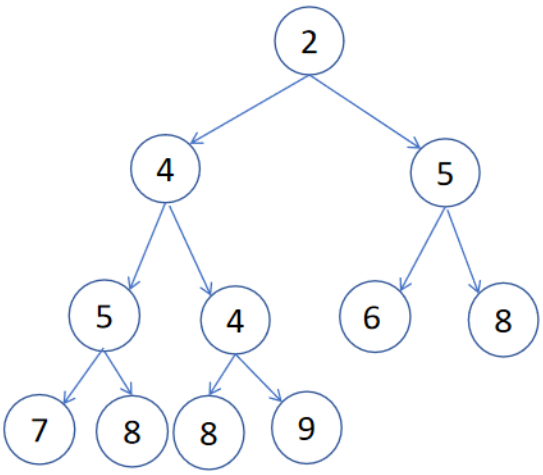
1. IDA*
2. RBFS
3. MA*
4. SMA*

Iterative Deepening A*

The simplest way to reduce memory requirements for A∗is to adapt the idea of iterative deepening. This search combines depth first search with incrementally increasing f-cost limits. The main difference between IDA* and standard iterative deepening is that the cutoff used is f-cost (g +h) rather than the depth; at each iteration, the cutoff value is the smallest f-cost of any node that exceeded the cutoff on the previous iteration.

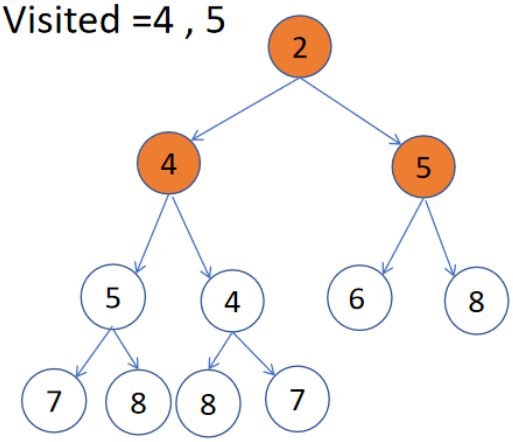
Example 1:

Start from 2 and reach 6 by applying Iterative Deepening Search. Note: f-score is written inside the nodes.



Iteration 1

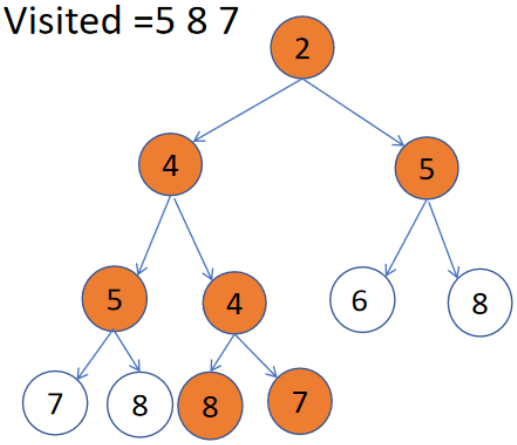
- Threshold = 2
- Visited =4 , 5



- Root node : 2
- Threshold = current node value, explore its children
- 4, 5>Threshold
- Prune values 4 and 5
- Threshold =4 (least values of pruned values)

Iteration 2

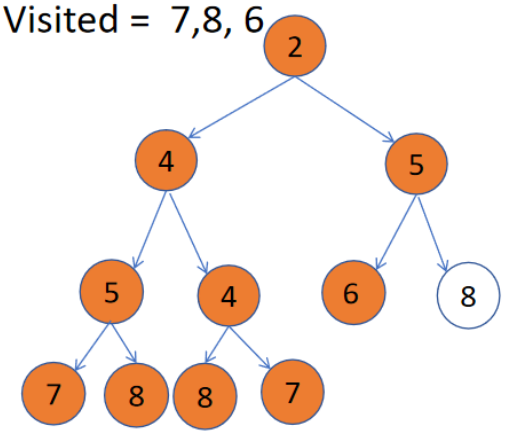
- Threshold = 4
- Visited =5 8 7



- Root node : 2 < Threshold, explore its children
- 4= Threshold, explore
  - 5>Threshold, prune
  - 4=Threshold, explore the child
    - 8, 9>Threshold, prune
- 5>Threshold
- Pruned values 5, 8, 7
- In pruned values, the least is 5, So threshold = 5

Iteration 3

- Threshold = 5
- Visited = 7,8, 6



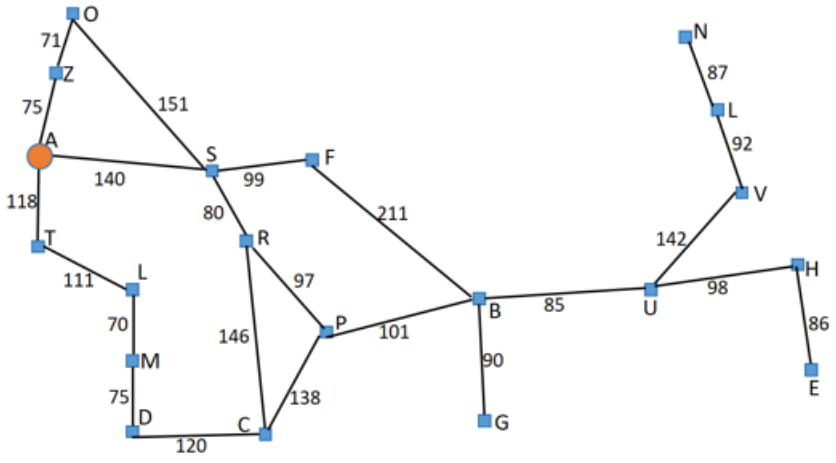
- 2 < threshold, explore its children
- First child 4 < threshold explore its children
  - 5= threshold explore its child
    - 7 , 8 > threshold, prune it
  - second child 4 < threshold, explore its children
    - 8,7 > threshold, prune it.
- Explore the second child of root node 2
  - 5 = threshold, explore its children
- Reached 6- goal state acheived

Recursive Best First Search (RBFS)

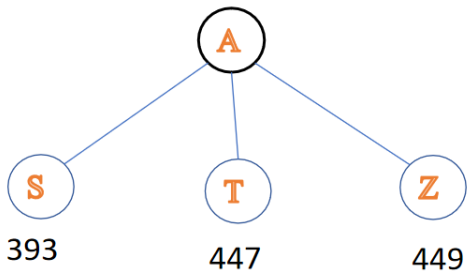
RBFS keep track of the f-value for best alternative path available from any ancestor of the current node. The f-limit value for each recursive call is shown on top of each current node, and every node is labeled with its f-cost. As the recursion is done, RBFS replaces the f-value of each node along the path with a backed-up value—the best f-value of its children. RBFS remembers f-value of the best leaf in the forgotten sub-tree and decide path.

Example 1:

For the given graph, travel from A to B applying Recursive Best First Search. The straight Line distance is given.



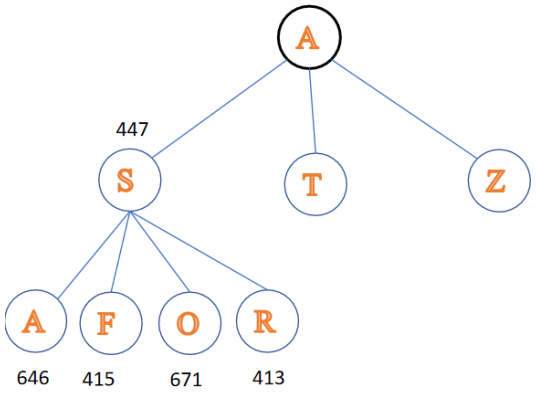
Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- A connected with S- T - Z
- f(n)= g(n)+h(n)
- value for
  - AS = 393 (140+253) - Shortest dist
  - AT = 447 (118+329) - f-limit
  - AZ = 449 (75+374)
- Best distance : AS
- Node to visit : S

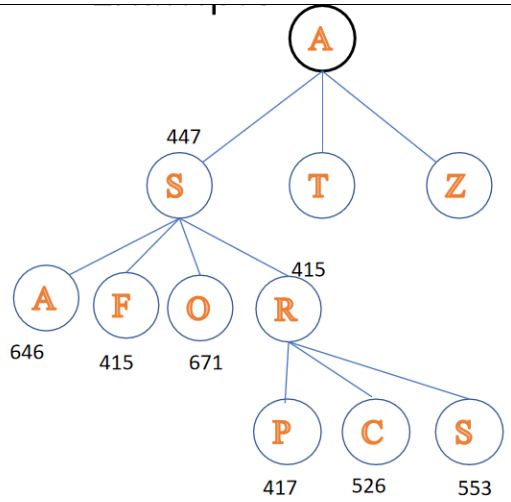
Visit A and explore all nodes and start expanding with the minimum distance



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- S connected with F-R-A-O
- f(n)= g(n)+h(n)
- value for
  - ASA = 646 (140+140+366)
  - ASF = 415 (140+99+176)
  - ASO = 671 (140+151+380)
  - ASR = 413 (140+80+193) Shortest dist
- Best Distance : ASR
- Node to visit : R

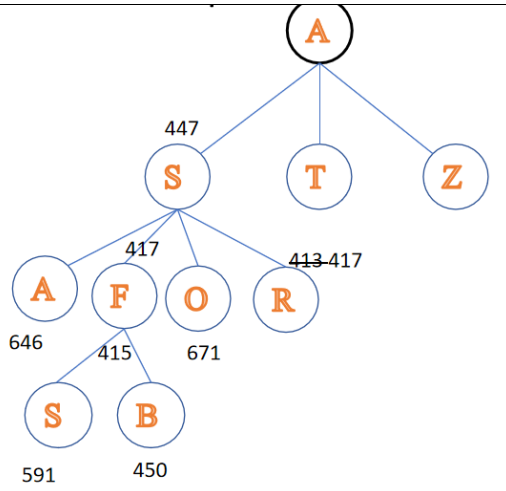
Expand S and calculate the distances for the explored node. The least distance is ASR. So the next node to visit is R



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- R is followed until P has a value that is worse than the best alternative path F
- R connected with P-C-S
- value for
  - ASF = 415 (140+99+176)
  - ASRS = 553 (140+80+80+253)
  - ASRC = 526 (140+80+146+160)
  - ASRP = 417(140+80+97+100)
- Best distance : ASF
- Node to visit : F

Expand R and calculate the connected nodes distance. Find the least distance node and it is F.

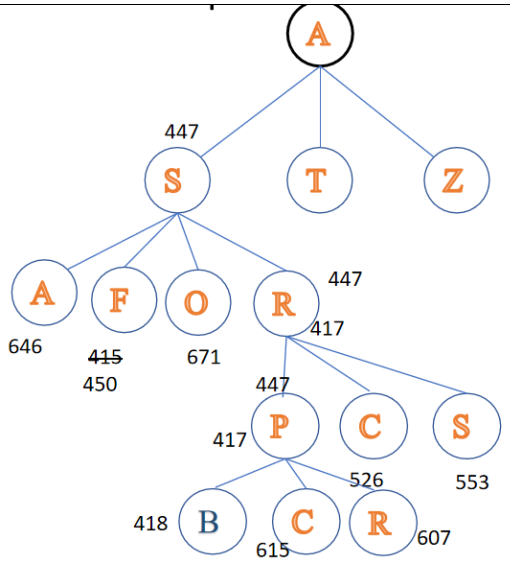


Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- The recursion unwinds and the best leaf value of the forgotten subtree (417) is backed up to (R)
- F connected with S- B
- value for
  - ASFB = 450 (140+99+211+0)
  - ASFS = 591(140+99+99+253)

  Goal State reached
- Shortest distance : ASFB (goal)
- Node to visit : P

Expand F. The previous node distance calculation expansion R is lesser compared to S and B. So expand R and keep the distance calculated in memory



Straight line distance to B	
A	366
B	0
C	160
D	242
E	161
F	176
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	100
R	193
S	253
T	329
U	80
V	199
Z	374

- The recursion unwinds, 450 is backed up to F then R is expanded.
- Best alternative path (through T) costs at least 447, the expansion continues to B.
- P connected with R- B- C
- value for
  - ASFB = 450(140+99+211+0)
  - ASRPB = 418(140+80+97+101+0)
  - ASRPC = 615(140+80+97+138+160)
  - ASRPR = 607(140+80+97+97+193)
- Shortest distance : ASRPB
- Node to visit : B Goal acheived

Expand P and the goal state can be achieved

Difference between IDA* and RBFS

	IDA*	RBFS
Memory	Uses too little memory	
Between Iterations	Retains Single number	Retains more information in memory but use only linear space
Expansion of nodes	Both algorithms expand the same states many times over	
Complexity	Exponential Increase	

MA* Search

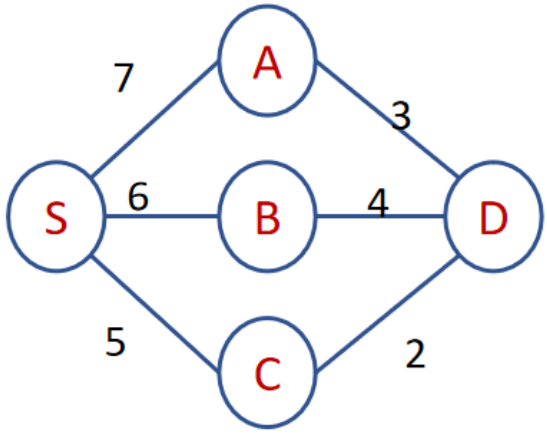
MA* search works like A* as long as sufficient memory remains. If no memory exists , MA* drops high f-cost leaf nodes, freeing memory. Since A* occupies more memory, MA* tries to overcome the defect of A* search by freeing memory. To prevent infinite looping, f-cost values are backed up through the tree. MA* is efficient search with the guarantee that complicated state spaces will not exceed system memory. MA* lead to the development of SMA*.

Simplified MA*

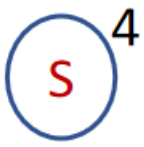
SMA* uses a binary tree to store the open list, while MA* uses a less efficient data structure. SMA* simplifies the storage of f-cost by eliminating the need for four f-cost quantities (as in MA*). SMA* performs the backup procedure when a node is fully expanded, instead of once for each generated node. SMA* also simplifies pruning by removing only enough nodes to remain below the memory limit, instead of pruning many unpromising nodes at a time.

Example 1:

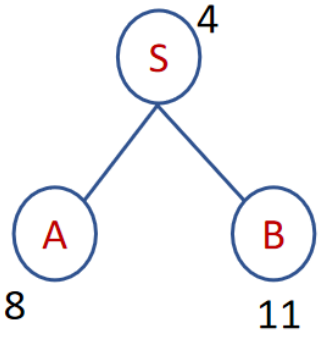
Start from S and reach D, applying SMA* with memory of 3 nodes.



Nodes	Hueristics
S	4
A	1
B	5
C	4
D	0

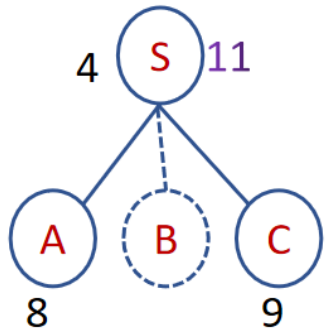


Start with the root node. Calculate f(n)=g(n)+h(n) = 0+4=4. Explore S, since binary tree only 2 nodes to be explored.



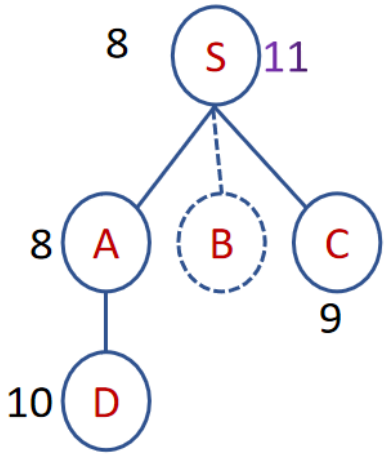
- SA= 7+1 =8
- SB= 6+5 =11

Explore S and find the distance with all nodes connected with S



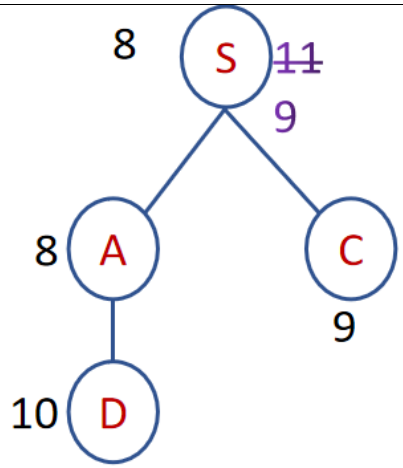
- SA= 7+1 =8
- ~~SB= 6+5 =11~~
- SC= 5+4 =8
- Memory Full
- Children with highest f value removed but value remembered by parent

Checking SB having the highest f-value. It is removed from the list but remembered by the parent



- Update S with lowest f value
- Explore A
- Visit D (Goal Node)

Explore A and visit D which is the goal node



- Explore other possibilities
- C<B, so update f value and forget B value
- All nodes explored
- Path to goal is S-A-D